

بسم الله الرحمن الرحيم



پروژه کارشناسی

موضوع : کدهای تحلیل FFT برای میکروکنترلر STM32F103

استاد گرامی : دکتر کریمیان

تهیه و تنظیم : زهرا نجف پور

فهرست

مقدمه.....	2
رond کلی پروژه	2
فوريه وتحليل FFT	3
الگوريتم روش بازگشتی	4
پياده سازی تبدیل فوريه بازگشتی به زبان C.....	10
تنظیمات stmcube و کدهای مربوطه.....	12
منابع	15

مقدمه

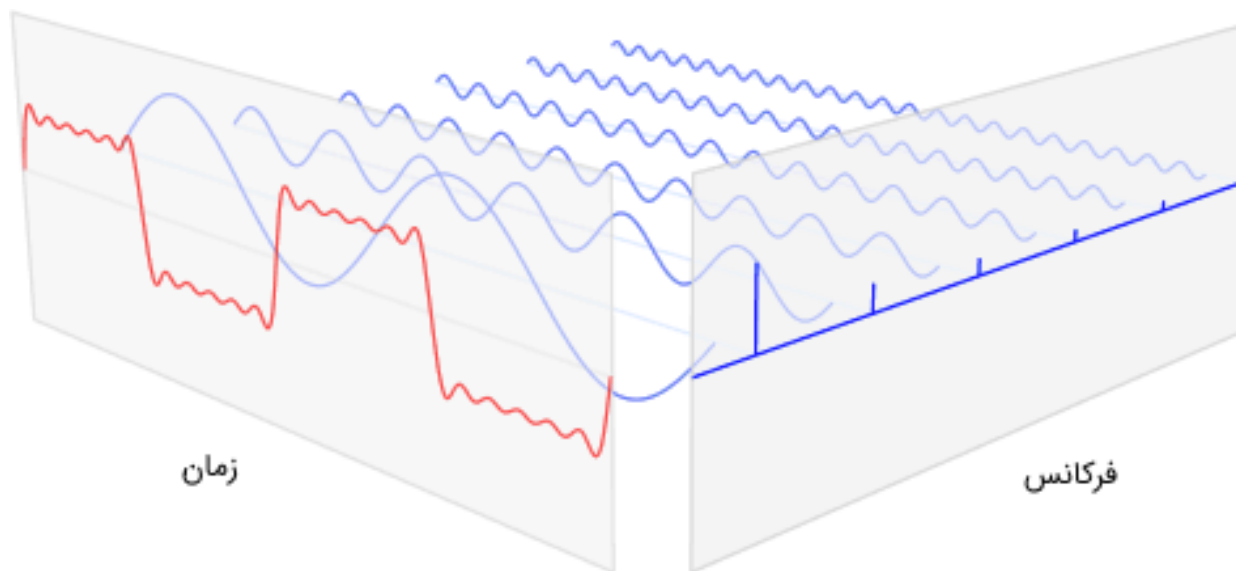
هدف از ارائه کدهای FFT برای میکروکنترلر STM32F103 در این پروژه ، به کارگیری آن در تشخیص قطعی مسیر جریان برای موتورهای قفس سنجابی میباشد ؛ بدین صورت که اگر برای تشخیص قطعی ، سیگنالی با فرکانس مشخص به موتور اعمال کنیم با توجه به وجود نویز و سایر سیگنالهای اضافی ، خروجی همان سیگنال ورودی نخواهد بود و تحلیل زمانی کمکی به تشخیص نخواهد کرد ، اما پس از گرفتن تبدیل فوریه در صورتیکه فرکانس ورودی اعمال شده موجود باشد ، آن فرکانس مشاهده میشود و در صورت عدم وجود فرکانس ورودی قطعی مسیر تشخیص داده میشود .

روند کلی پروژه

روند کلی بدین صورت است که با استفاده از میکرو stm32f103 ، تنظیمات ADC را انجام داده و سیگنال آنالوگ ورودی را نمونه برداری میکنیم و با در دست داشتن سیگنال گسسته ، با استفاده از تحلیل FFT ، سیگنال نمونه برداری شده را درحوزه فرکانس مشاهده میکنیم .

فوريه و تحليل FFT

آناليز فوريه مي تواند يك سيگنال از حوزه اصلي، كه معمولا زمان يا فضا است را به نمايشي در حوزه فرکانس و نيز بلعكس تبديل كند. تبديل فوريه گسسته معمولا از طريق تجزيه دنباله مقادير، به عناصر با فرکانس هاي متفاوت محاسبه مي شود. اين تبديل در بسياري از رشته ها مفيد است، اما مشكلي كه وجود دارد اين است كه محاسبه مستقيم اين تبديل با استفاده از تعريف آن بسيار كند است و در عمل كاربردي ندارد؛ فوريه سريع يا FFT روشي است كه به وسيله آن مي توان تبديل فوريه گسسته را به سرعت محاسبه كرد.



افرادى به نام هاي كولي و توكي (Cooley and Tukey) توانستند الگوريتمي براي محاسبه تبديل فوريه سريع يا Fast Fourier Transform به دست بياورند. در اين الگوريتم كه مهم ترين الگوريتم تبديل فوريه سريع است، به صورت بازگشتي (Recursively) تبديل فوريه گسسته را به مساييل كوچك تر مي شكند و زمان مورد نياز براي انجام محاسبات را به مقدار قابل توجهي كاهش مي دهد.

الگوریتم روش بازگشتی

فرمول تبدیل فوریه گسسته به شکل زیر بیان میشود :

$$X_k = \sum_{n=0}^{N-1} x_n e^{\frac{-i2k\pi n}{N}}$$

و همچنین تبدیل فوریه گسسته معکوس به صورت زیر است :

$$X_k = \frac{1}{N} \sum_{n=0}^{N-1} X_k e^{\frac{i2k\pi n}{N}}$$

که N تعداد نمونه ها و n نمونه فعلی است . همچنین x_n مقدار سیگنال در زمان n و k فرکانس فعلی و X_k نتیجه حاصل فوریه است .

حال می‌خواهیم تعداد ضرب و جمع‌های مورد نیاز برای محاسبه تبدیل فوریه گسسته با استفاده از فرمول بالا را به دست آوریم ؛ برای هر ضریب در تبدیل فوریه X_k باید عبارات را که شامل $x(0)e^{-j\frac{2\pi}{N}k \times 0}$ و $x(1)e^{-j\frac{2\pi}{N}k \times 1}$ تا $x(N-1)e^{-j\frac{2\pi}{N}k \times (N-1)}$ هستند را محاسبه کنیم و سپس این عبارات را باهم جمع کنیم .

در حالت کلی عدد مختلط است و هر خروجی تبدیل فوریه گسسته تقریباً به N ضرب مختلط و $N-1$ جمع مختلط نیاز دارد . یک ضرب مختلط به تنهایی به 4 ضرب حقیقی و 2 جمع حقیقی نیاز دارد ، بنابراین برای محاسبه تبدیل فوریه نهایی لازم است تا DFT را برای تمام N مقدار از K محاسبه کنیم .

بنابراین آنالیز تبدیل فوریه N نقطه به $4N^2$ جمع حقیقی و $N(4N-2)$ جمع حقیقی نیاز دارد . بنابراین محاسبات از درجه N^2 خواهد بود و با افزایش N محاسبات مورد نیاز به سرعت افزایش میابد .

برای کاهش محاسبات روش بازگشتی را داریم که پیچیدگی مسئله از درجه N^2 به $N \log N$ کاهش میابد.

یکی از مهم ترین ابزارها در ساخت یک الگوریتم در برنامه نویسی این است که تقارن را در حل مسئله اعمال

کنیم. اگر به صورت تحلیلی بتوان نشان داد که یک قسمت از مسئله به قسمت دیگری از آن مشابه است،

می توان به سادگی آن قسمت را یک بار محاسبه کرد و هزینه محاسباتی را کاهش داد.

مقدار X_{N+k} با توجه به فرمول تبدیل فوریه گسسته به صورت زیر است :

$$\begin{aligned} X_{N+k} &= \sum_{n=0}^{N-1} x_n e^{i \frac{2\pi(N+k)n}{N}} \\ &= \sum_{n=0}^{N-1} x_n e^{-i2\pi n} \cdot e^{\frac{-i2\pi kn}{N}} \\ &= \sum_{n=0}^{N-1} x_n e^{\frac{-i2\pi kn}{N}} \end{aligned}$$

در این فرمول از خاصیت همانندی (Identity) استفاده شده است ، خط آخر در فرمول بالا، مشخصه تقارن بسیار مهمی را در تبدیل فوریه گسسته نشان می دهد که در نتیجه آن داریم:

$$X_{N+k} = X_k$$

که به سادگی میتوان نوشت :

$$X_{iN+k} = X_k$$

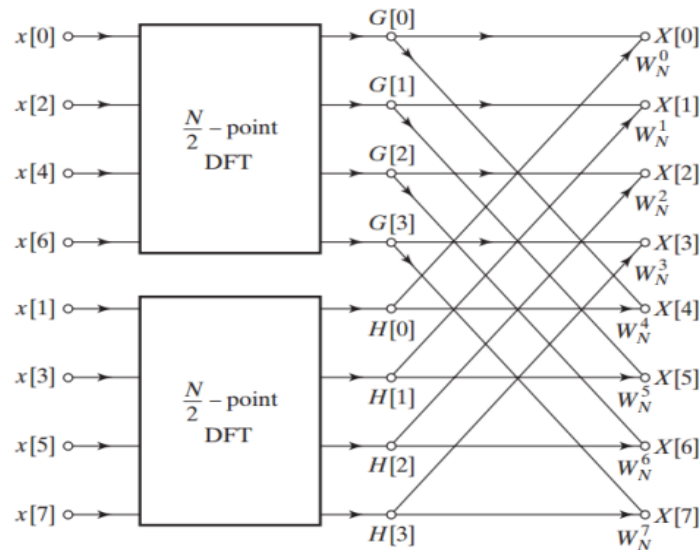
با استفاده از تعریف تبدیل فوریه گسسته به صورت زیر سعی میکنیم طبق روش بازگشتی به دو قسمت کوچکتر یعنی مقادیر زوج و فرد آنرا بشکنیم :

$$\begin{aligned}
X_k &= \sum_{n=0}^{N-1} x_n e^{\frac{-i2k\pi n}{N}} \\
&= \sum_{m=0}^{\frac{N}{2}-1} x_{2m} e^{\frac{-i2k\pi(2m)}{N}} + \sum_{m=0}^{\frac{N}{2}-1} x_{2m+1} e^{\frac{-i2k\pi(2m+1)}{N}} \\
&= \sum_{m=0}^{\frac{N}{2}-1} x_{2m} e^{\frac{-i2k\pi m}{\frac{N}{2}}} + e^{-i\frac{2\pi k}{N}} \sum_{m=0}^{\frac{N}{2}-1} x_{2m+1} e^{\frac{-i2k\pi m}{\frac{N}{2}}}
\end{aligned}$$

در این حالت، تبدیل فوریه گسسته تکی را به دو عبارت تقسیم کردیم که هر کدام شباهت بسیار زیادی به عبارت تبدیل فوریه اصلی دارند و یکی بر روی اعداد فرد و دیگری بر روی اعداد زوج عمل می کنند. اما تا این قسمت هنوز هیچ توان محاسباتی را کاهش نداده ایم و هر عبارت از $\frac{N}{2} * N$ محاسبه تشکیل شده است که در مجموع N^2 محاسبه را شامل می شود.

آنچه موجب کاهش هزینه محاسباتی می شود، استفاده از خاصیت تقارن در هر یک از عبارت های محاسبه شده است. زیرا بازه K برابر با $0 \leq k \leq N$ است، در حالی که بازه n برابر با $0 \leq n \leq M \equiv \frac{N}{2}$ در نظر گرفته می شود. در خاصیت تقارن بالا نکته ای که وجود دارد این است که می توان برای هر زیر مسئله فقط نیمی از محاسبات را انجام داد. بنابراین محاسبات N^2 تبدیل به M^2 می شود که M برابر با نصف اندازه N است. اما روند کار در اینجا متوقف نمی شود. تا زمانی که تبدیل فوریه کوچک تر دارای مقدار M زوج باشد، می توانیم این روش تقسیم و غلبه (Divide-and-Conquer) را به صورت تکراری انجام دهیم و هر بار هزینه محاسباتی را نصف کنیم. این روند را تا جایی ادامه می دهیم که آرایه به دست آمده آنقدر کوچک باشد که استفاده از این استراتژی دیگر تاثیری در بهبود محاسبات نداشته باشد.

تصویر زیر نمایی از تجزیه تبدیل فوریه گسسته ۸ نقطه‌ای به دو تبدیل فوریه ۴ نقطه‌ای را نشان می‌دهد. در این نمودار $W_N^k = e^{-j\frac{2\pi}{N}k}$ است.



تجزیه تبدیل فوریه گسسته ۸ نقطه‌ای به دو تبدیل فوریه ۴ نقطه‌ای

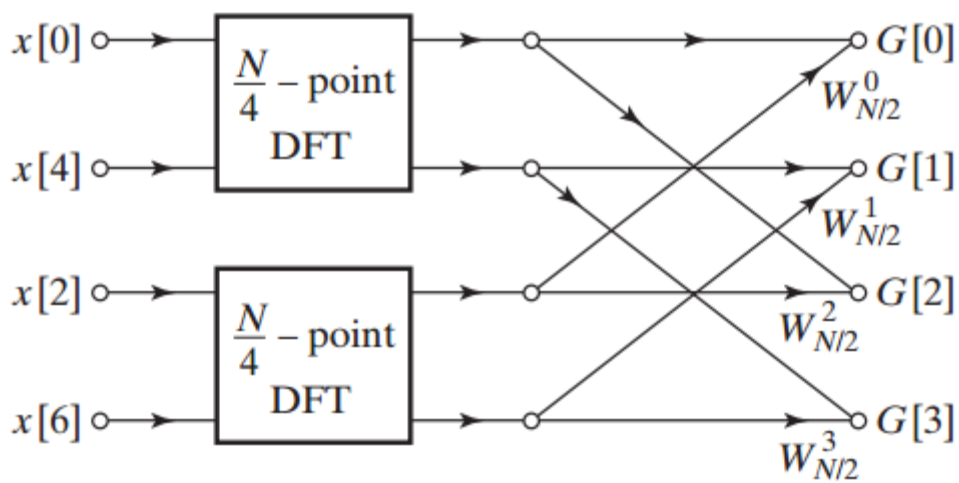
زمانی که از معادله اصلی تبدیل فوریه گسسته استفاده می‌کنیم، نیاز است تا از $4N^2 = 256$ ضرب حقیقی برای هشت نقطه استفاده کنیم، در حالی که با الگوریتم توضیح داده شده، می‌توان دو تبدیل فوریه ۴ نقطه‌ای را با استفاده از N ضرب مختلط و $2 \times 4 \times \left(\frac{N}{2}\right)^2 = 2 \times 4 \times 4^2 = 128$ ضرب حقیقی محاسبه کرد. N ضرب مختلط خود به $4N = 4 \times 8 = 32$ ضرب حقیقی نیاز دارند. بنابراین الگوریتم در حالت کلی به ۱۶۰ ضرب حقیقی نیاز دارد. این عدد در مقایسه با ۲۵۶ ضرب حقیقی در حالت قبلی، کمتر است. این بهبود قابل توجهی است، اما می‌توان از طریق اعمال روش یاد شده به DFT چهار نقطه‌ای به بهبود بیشتری نیز دست یافت.

دیدیم که نقطه شروع الگوریتم، طول DFT است که زوج در نظر گرفته می‌شود و ما قادریم که عبارت نمایی مختلط از اندیس زوج را ساده کنیم. به همین دلیل، برای اینکه بار دیگر الگوریتم را ساده‌سازی کنیم، نیاز

داریم که N^2 مضربی از دو باشد. در این مثال خاص $\frac{N}{2} = 4$ است و با استفاده از الگوریتم بالا می‌توانیم هر

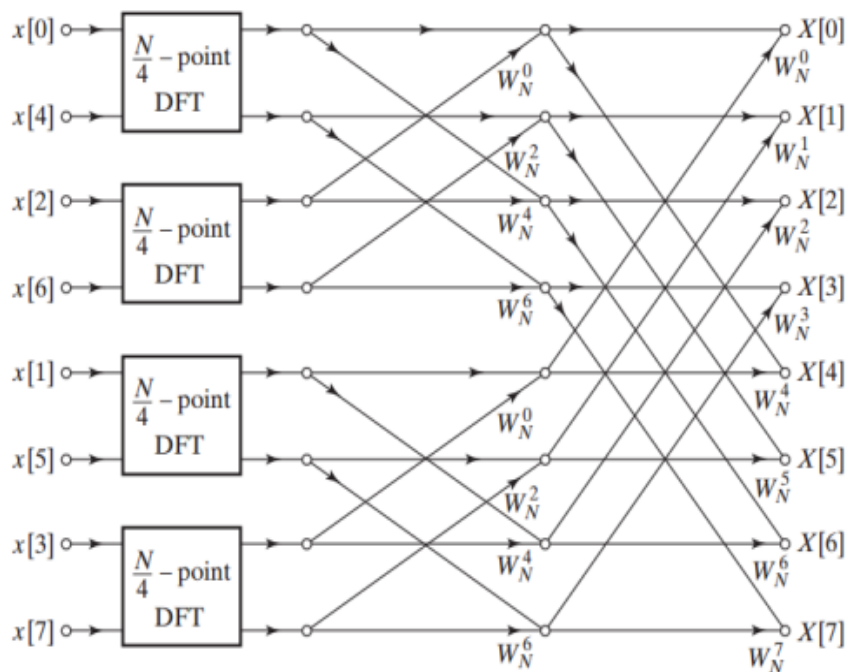
کدام از دو تبدیل فوریه ۴ نقطه‌ای را به دو تبدیل فوریه ۲ نقطه‌ای تجزیه کنیم. به عنوان مثال، تبدیل فوریه

گسسته N^2 نقطه اولی در نمودار شکل بالا را می توان به صورت زیر نشان داد.

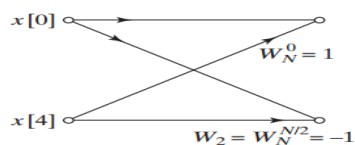


شکستن تبدیل فوريه گسسته $\frac{N}{2}$ نقطه در نمودار قبل

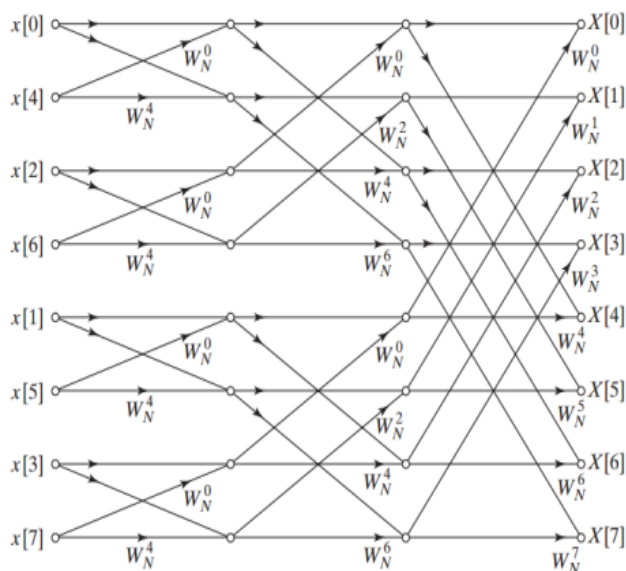
با جایگذاری نمودار بالا در هر کدام از تبدیل فوريه های گسسته ۴ نقطه ای، می توان به نمودار جدید زیر رسید.



سرانجام، می‌توانیم تبدیل فوریه دو نقطه‌ای در تصویر بالا را نیز با نمودار زیر جایگزین کنیم.



بعد از جایگذاری، در نهایت به نمودار زیر می‌رسیم.



در مثال بررسی شده، نشان داده شد که به منظور پیاده‌سازی تکنیک به صورت تکراری، تا زمان باقی ماندن تبدیل فوریه گسسته دو نقطه‌ای، باید طول تبدیل فوریه گسسته اصلی را از توان دو انتخاب کنیم. به عبارت دیگر باید $N = 2^v$ برقرار باشد. در این حالت پس از $\log_2(N)$ تکرار، به دو تبدیل فوریه گسسته دو نقطه‌ای می‌رسیم. همان طور که از تصویر آخر نیز می‌توان مشاهده کرد، هر گام از الگوریتم به $N \log_2(N)$ ضرب مختلط نیاز دارد. مثلاً در مورد تبدیل فوریه هشت نقطه‌ای $8 \log_2(8) = 24$ ضرب مختلط و $24 \times 4 = 96$ ضرب حقیقی نیاز داریم، در حالی که محاسبه مستقیم این DFT به ۲۵۶ ضرب حقیقی نیاز دارد. بنابراین در محاسبه تبدیل فوریه گسسته برای داده‌های طولانی، این الگوریتم می‌تواند منجر به کاهش محاسبات به اندازه قابل توجهی شود.

پیاده سازی تبدیل فوریه بازگشتی به زبان C

```
typedef struct {
    int32_t real;
    int32_t imag;
}
fft_i32_t;

void core_fft_i32(fft_context_i32_t* context, fft_i32_t * dest, fft_i32_t * src, int sign){
    int n, k, m;
    int half_N;
    int k2;
    int twiddle;
    int twiddle_jump;
    int twiddle_max;
    int n_jump;
    int n_start;
    fft_i32_t tmp_product;
    fft_i32_t twiddle_factor;
    fft_i32_t tmp;

    //copy the signal input to the tmp variable
    tmp=input;
    for(n=0; n < context->N; n++){
        m = bit_reversal(n, context->order);
        dest[n] = src[m];
    }

    //This loop performs the FFT using the Cooley-Tukey algorithm
    half_N = context->N >> 1;
    for(k=0; k < context->order; k++){
        n = 0;
        k2 = (1<<k);

        //Calculate the twiddle jump and max for the stage
        twiddle_max = (1<<(context->order - 1));
        twiddle_jump = (1<<(context->order - k - 1)); //equals 2^(kmax - k)

        //Calculate the n jump for the stage
        n_jump = k2<<1;
        n_start = 0;
```

```

135 half_N = context->N >> 1;
136 for(k=0; k < context->order; k++){
137     n = 0;
138     k2 = (1<<k);
139
140     //Calculate the twiddle jump and max for the stage
141     twiddle_max = (1<<(context->order - 1));
142     twiddle_jump = (1<<(context->order - k - 1)); //equals 2^(kmax - k)
143
144     //Calculate the n jump for the stage
145     n_jump = k2<<1;
146     n_start = 0;
147
148     for(twiddle = 0; twiddle < twiddle_max; twiddle += twiddle_jump){
149
150         //twiddle value in the table (always < N/2)
151         twiddle_factor.real = context->twiddle_table[twiddle].real;
152         twiddle_factor.imag = context->twiddle_table[twiddle].imag * sign;
153
154         //This loop executes a single butterfly operation (size-2 DFT operation)
155         for(n=n_start; n < context->N; n+= n_jump){
156             m = n+k2;
157             //This is the butterfly code
158             tmp_product = fft_mult_132(dest[m], twiddle_factor); //multiply
159             tmp = scale_add_132(dest[n], tmp_product, context->scale); //accumulate
160             dest[m] = scale_subtract_132(dest[n], tmp_product, context->scale);
161             dest[n] = tmp;
162
163         }
164         n_start++;
165     }
166 }
167 }
168 }
169 }
170 }
171 /* USER CODE BEGIN 3 */
172 HAL_ADC_Stop(&hadcl);
173
174

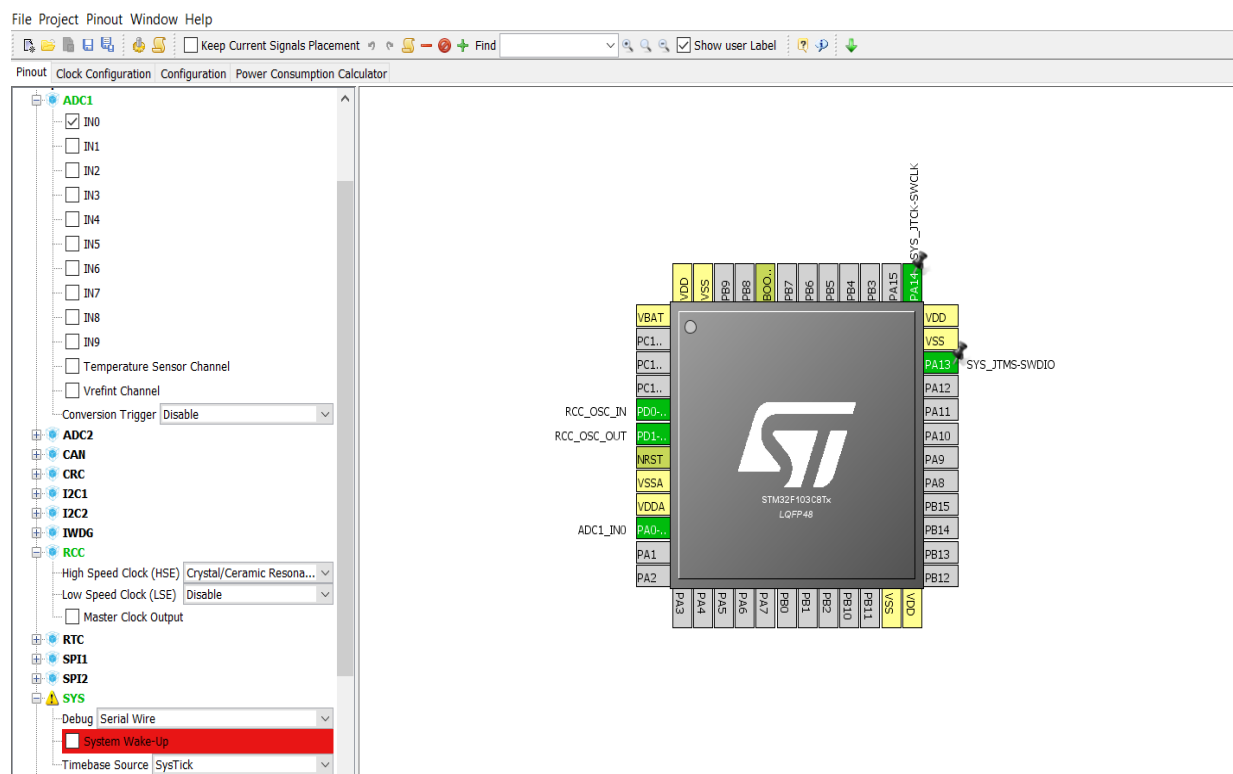
```

توضیحات بخش تنظیمات نرم افزار stmcube و کدنویسی برای واحد ADC

برد انتخاب شده برای پروژه ، برد دیسکاواری stm32f103cxx میباشد ، این برد دارای حداکثر فرکانس 72MHz و 2 واحد ADC با دقت 12 بیتی است .

برای برای پیشبرد روند کاری با استفاده از نرم افزار stmcube ابتدا تنظیمات ADC را با توضیحات زیر انجام داده و کدهای مورد نیاز را تولید میکنیم :

برای این منظور از بین 2 واحد 12 پایه ای واحد ADC ما گزینه IN0 از ADC1 را انتخاب میکنیم



برای تولید فرکانس از کریستال خارجی استفاده میکنیم و تنظیمات مربوطه را با توجه به اینکه فرکانس

IN0 از واحد ADC1 توسط باس APB2 تامین میشود ، انجام میدهیم .

در قسمت clock configuration نرم افزار پس از فال کردن HSE مشاهده میکنیم حداکثر فرکانسی که ADC میتواند با آن کار کند 12MHz میباشد.

از راه های نمونه برداری scan conversion mode انتخاب میکنیم ، سپس در قسمت Rank ، chanel0 را ، با توجه به هدف ، فرکانس 12MHz برای فراهم کردن حداکثر نمونه در ثانیه، 1.5 سیکل را انتخاب میکنیم که طبق رابطه میشود 860000 نمونه در هر ثانیه .

طبق نایکوئیست :

$$\frac{f_{ADC}}{cycle_{total}} = \text{تعداد نمونه ها}$$

برای برنامه ریزی میکرو نیز از روش ST-Link استفاده میکنیم که به این منظور تنظیمات , SWDIO

SWCLK را نیز در نرم افزار stmcube در نظر میگیریم تا کدهای لازم تولید شوند .

پس از تولید کدها توسط این نرم افزار وارد نرم افزار keil میشویم ، در فایل main.c

یک استراکت توسط stmcube تولید شده که در توابع به عنوان ورودی استفاده میکنیم و سایر کدهای

مربوطه قابل مشاهده است .

```

hadcl.Init.NbrOfConversion = 2;
if (HAL_ADC_Init(&hadcl) != HAL_OK)
{
    Error_Handler();
}

/**Configure Regular Channel
*/
sConfig.Channel = ADC_CHANNEL_0;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE
if (HAL_ADC_ConfigChannel(&hadcl, &sConfig)
{

```

برای کدهای مربوط به حلقه while سیگنال سنس شده از ورودی را دریافت کرده و پس از عبور از واحد ADC با کدهای زیر :

کدها شامل مراحل (1 انتخاب رنک مورد نظر 2) Initona کردن (3 مرحله استارت و قرار دادن شرطی برای اطلاع از تبدیل 4) مقدار را خوانده داخل متغیر input قرار مید دهیم .

```

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    hadcl.Init.NbrOfConversion=1;
    HAL_ADC_Init(&hadcl);
    HAL_ADC_Start(&hadcl);
    if (HAL_ADC_PollForConversion(&hadcl,300)==HAL_OK)
    {
        input=HAL_ADC_GetValue(&hadcl);
    }
/* USER CODE END WHILE */

    typedef struct {
        int32_t real;
        int32_t imag;
    };

```

در کد تبدیل فوریه سریع متغیر input را داخل متغیر tmp قرار میدهد و پس از انجام محاسبات آنرا تحویل میدهد ؛ برای اینکار با اضافه کردن کتابخانه DSP و math تحلیل رو انجام میدیم . در آخر adc را stop میکنیم .

```

        tmp_product = fft_mult_i32(dest[m], twiddle_factor); //multiply
        tmp = scale_add_i32(dest[n], tmp_product, context->scale); //accumulate
        dest[m] = scale_subtract_i32(dest[n], tmp_product, context->scale);
        dest[n] = tmp;

        n_start++;
    }
}
}

/* USER CODE BEGIN 3 */
HAL_ADC_Stop(&hadcl);

/* USER CODE END 3 */

/** System Clock Configuration
*/
void SystemClock_Config(void)
{

```

- 1) <https://stm32f4-discovery.net/2014/10/stm32f4-fft-example/>
- 2) <https://community.arm.com/developer/tools-software/tools/f/armds-forum/2358/512-bit-fft-on-arm-cortex-m3>
- 3) https://www.keil.com/pack/doc/CMSIS/DSP/html/group_RealFFT.html
- 4) https://github.com/MaJerle/stm32fxxx-hal-libraries/blob/master/14-STM32Fxxx_FFT/User/main.c
- 5) **Inside the FFT Black Box: Serial and Parallel Fast Fourier Transform Algorithms**